EECS 182: Deep Learning Review Guide

Homework 12 Preparation: VAEs, Diffusion, & Meta-Learning

Estimated Time: 30-45 minutes

Part 1: Key Concepts Review (10-15 minutes)

1.1 Variational Autoencoders (VAEs)

Core Idea: VAEs are generative models that learn to encode data into a latent space and decode it back. Unlike standard autoencoders, VAEs learn a *probabilistic* mapping.

Architecture:

- **Encoder $q_\phi(z|x)$:** Maps input x to latent distribution parameters ($\mu_\phi(x)$, $\sigma_\phi(x)$)
- **Latent Space:** Sample $z \sim N(\mu_\phi(x), \sigma_\phi(x))$
- **Decoder $p_\theta(x|z)$:** Reconstructs input from latent code

The Reparameterization Trick:

To enable backpropagation through sampling: $z = \mu + \sigma \blacksquare \varepsilon$, where $\varepsilon \sim N(0, I)$

Loss Function (ELBO):

$$L_{VAE} = E_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \,||\, p(z))$$

- **First term:** Reconstruction loss (how well can we reconstruct x from z)
- **Second term:** Regularization (keeps latent distribution close to prior $p(z) = N(0, I)$)

Why the KL term matters: It creates the information bottleneck—forcing the model to learn compressed, meaningful representations rather than memorizing.

1.2 KL Divergence

Definition: $D_{KL}(P||Q) = E_{x\sim P}[\log P(x)/Q(x)]$

Key Properties:

- **Asymmetric:** $D_{KL}(P||Q) \neq D_{KL}(Q||P)$ in general
- **Non-negative:** Always ≥ 0, equals 0 only when P = Q
- **Forward KL - $D_{KL}(P||Q)$:** "Mean-seeking" - Q tries to cover all of P
- **Reverse KL - $D_{KL}(Q||P)$:** "Mode-seeking" - Q focuses on high-probability regions of P

Intuition: Forward KL penalizes Q for having low probability where P has high probability. Reverse KL penalizes Q for having high probability where P has low probability. This leads to different behaviors when Q cannot perfectly match P.

1.3 Diffusion Models (Brief Overview)

Basic Idea: Learn to reverse a gradual noising process.

- **Forward Process:** Gradually add noise: $x_0 \to x_1 \to ... \to x_T$ (pure noise)
- **Reverse Process:** Learn $p_\theta(x_{t-1}|x_t)$ to denoise
- **Training:** Learn to predict the noise or the denoised image at each step
- **Generation:** Start from pure noise, iteratively denoise

Key Insight: At each timestep t, we can jump directly to x_t without computing intermediate steps— this makes training efficient using the reparameterization trick.

1.4 Meta-Learning & MAML

Goal: Learn how to learn—find initializations that adapt quickly to new tasks with minimal data.

MAML Algorithm:

1. **Sample tasks** T_i from task distribution
2. For each task, perform **inner loop** updates (few-shot learning):
   $\theta'_i = \theta - \alpha \nabla_\theta L_{T_i}(\theta)$
3. Compute **meta-gradient** using task-specific parameters on test data:
   $\theta \leftarrow \theta - \beta \nabla_\theta \Sigma_{T_i} L_{T_i}(\theta'_i)$

Two types of parameters:

- **Slow weights (θ):** Meta-learned initialization, updated across tasks
- **Fast weights (θ'):** Task-specific, adapted within each task

Part 2: Important Discussion Problem Patterns (5 minutes)

2.1 Diffusion Models - Key Insights

• **Binary Pixel Flipping:** Probability that a pixel hasn't been replaced after T steps: $(1 - 2\omega)^T$
• **Fast Sampling at Time t:** No need to generate all intermediate steps—directly sample using the probability that a pixel hasn't been replaced: $(1 - 2\omega)^t$
• **Activation for Probabilities:** Sigmoid is the correct choice (outputs [0,1]). ReLU can exceed 1, tanh can be negative.

2.2 VAE Training and Inference

• **Training:** Use both encoder and decoder. Encoder provides $q_\phi(z|x)$, decoder reconstructs
• **Generation:** Only use decoder. Sample $z \sim N(0, I)$ from prior, then decode
• **Why Blurry?:** KL term forces low-frequency information, stochastic sampling averages over possibilities
• **Information Bottleneck:** Created by (1) smaller latent dimension and (2) KL regularization term

2.3 KL Divergence Behavior

• **Forward KL minimization:** Q tries to cover all modes of P (mean-seeking, may be diffuse)
• **Reverse KL minimization:** Q concentrates on one mode of P (mode-seeking, sharper but may miss modes)
• **In VAEs:** We use reverse KL, which encourages the approximate posterior to focus on likely regions

Part 3: Warmup Questions (15-25 minutes)

Work through these problems, then check your answers on the last page.

Question 1: VAE Components (5 minutes)

Consider a VAE with 2D latent space ($z \in \blacksquare^2$) trained on 28×28 MNIST images (784 pixels).

(a) If the encoder outputs μ = [1.5, -0.8] and log σ^2 = [0.2, -0.5], write the formula to sample z using the reparameterization trick with $\varepsilon \sim N(0, I)$.

(b) What are the dimensions of the decoder output if we model each pixel as an independent Bernoulli variable?

(c) Which term in the VAE loss encourages the latent codes to be normally distributed: reconstruction or KL term?

Question 2: KL Divergence Understanding (5 minutes)

Suppose P is a mixture of two Gaussians (bimodal), and Q is a single Gaussian.

(a) If we minimize $D_{KL}(P||Q)$ (forward KL), will Q tend to: (i) cover both modes broadly, or (ii) concentrate on one mode?

(b) If we minimize $D_{KL}(Q||P)$ (reverse KL), will Q tend to: (i) cover both modes broadly, or (ii) concentrate on one mode?

(c) Which divergence direction is used in the VAE ELBO?

Question 3: Computing VAE Loss (5-7 minutes)

Given a 1D latent space VAE where: • Encoder outputs: $\mu = 2.0$, $\sigma = 1.5$ • Prior: $p(z) = N(0, 1)$ • Sampled latent: $z = 3.0$ • True image pixel values: $x = [1, 0, 1]$ • Decoder outputs (Bernoulli parameters): [0.9, 0.1, 0.8]

(a) Calculate the reconstruction loss (negative log-likelihood) for this sample. Hint: For Bernoulli, $-\log p(x|z) = -\Sigma[x \cdot \log(p) + (1-x) \cdot \log(1-p)]$

(b) Calculate the KL divergence term. Hint: For Gaussians, $D_{KL}(N(\mu,\sigma^2) \,||\, N(0,1)) = 0.5 \cdot (\sigma^2 + \mu^2 - 1 - \log \sigma^2)$

Question 4: Meta-Learning Concepts (4-6 minutes)

In MAML for a 3-class classification problem:

(a) During the inner loop, we sample 5 examples from Task 1 and update: $\theta'_1 = \theta - \alpha\nabla L_{task1}(\theta)$. Do we compute gradients with respect to θ or θ'_1 during this step?

(b) In the meta-update (outer loop), we use test examples from the task to compute gradients. These gradients flow back through the inner loop updates. What parameters do we actually update in the meta-update step?

(c) Why might MAML be preferred over just training on all tasks simultaneously?

Question 5: Diffusion Sampling (3-5 minutes)

For a binary diffusion model with flip probability $\omega = 0.1$ per step:

(a) What's the probability that a pixel has NOT been replaced by random noise after $t = 3$ steps?

(b) If we want to sample Y_5 directly from Y_0 (skipping intermediate steps), what's the probability we keep the original pixel value?

(c) As $t \to \infty$, what distribution does Y_t approach?

Solutions

Check your work:

Solution 1:

(a) $z = \mu + \exp(0.5 \cdot \log \sigma^2)$ ■ $\varepsilon = [1.5, -0.8] + [\exp(0.1), \exp(-0.25)]$ ■ $\varepsilon = [1.5 + 1.105\varepsilon_1, -0.8 + 0.779\varepsilon_2]$

(b) 784 dimensions (one probability per pixel)

(c) The KL term $D_{KL}(q_\phi(z|x) \| p(z))$ encourages $q_\phi(z|x) \approx N(0, I)$

Solution 2:

(a) Forward KL is mean-seeking: Q will (i) cover both modes broadly to avoid being zero where P has mass

(b) Reverse KL is mode-seeking: Q will (ii) concentrate on one mode to avoid having mass where P is zero

(c) Reverse KL: $D_{KL}(q_\phi(z|x) \| p(z))$

Solution 3:

(a) Reconstruction loss:

$-[1\cdot\log(0.9) + 0\cdot\log(0.9) + 0\cdot\log(0.1) + 1\cdot\log(0.1) + 1\cdot\log(0.8) + 0\cdot\log(0.2)]$
$= -[\log(0.9) + \log(0.1) + \log(0.8)]$
$= -[-0.105 - 2.303 - 0.223] \approx 2.63$

(b) KL divergence:

$0.5\cdot(1.5^2 + 2.0^2 - 1 - \log(1.5^2))$
$= 0.5\cdot(2.25 + 4.0 - 1 - 0.811)$
$= 0.5\cdot(4.439) \approx 2.22$

Solution 4:

(a) We compute gradients with respect to θ (the original parameters) during the inner loop

(b) We update θ (the meta-learned initialization). The gradients flow backward through θ'_1 back to θ using the chain rule.

(c) MAML learns an initialization that's specifically good for quick adaptation to new tasks, rather than just averaging performance across all tasks. It optimizes for few-shot learning capability.

Solution 5:

(a) $(1 - 2\cdot0.1)^3 = 0.8^3 = 0.512$ (about 51%)

(b) $(1 - 2\cdot0.1)$■ = 0.8■ ≈ 0.328 (about 33%)

(c) Uniform distribution over {-1, +1} (i.e., fair coin for each pixel independently)

■■

You're ready for the homework! Good luck! ■